

Сортиране чрез сравнения във време $O(n \log n)$

0. Защо изобщо говорим за $O(n \log n)$?

Простите сортиращи алгоритми (Bubble Sort, Selection Sort, Insertion Sort) работят за $\Theta(n^2)$. Това е приемливо за хиляди елементи, но за милиони става безсмислено бавно. Има теоретична долна граница, която казва, че **никаой сортиращ алгоритъм, основан само на сравнения, не може да работи по-бързо от $\Omega(n \log n)$ в най-лошия случай**. Тоест $n \log n$ е „светлинната скорост“ за този клас алгоритми, и две елегантни идеи я достигат:

1. **HEAPSORT** — използва специална структура (пирамида), която ни дава достъп до максимума за $O(1)$ и поддръжка на структурата за $O(\log n)$ на операция.
2. **MERGESORT** — типичен **Разделяй-и-Владей** алгоритъм, който разделя задачата наполовина, решава рекурсивно и слива резултатите.

Двата алгоритъма имат еднаква асимптотика, но много различен дух — добре е да разбираш и двата, защото идеите им се срещат на всяка крачка после.

Част I. Двоична пирамида

1. Определение и свойства

1.1 Попълнено двоично дърво (complete binary tree)

Преди да говорим за пирамида, трябва да си наясно с **попълнено двоично дърво**. Това е наредено двоично дърво, в което:

- всички нива, **с възможно изключение на последното**, имат максималния възможен брой върхове (ниво k има точно 2^k върха);
- листата на последното ниво са **максимално вляво**.

С други думи: пълниш нивата от ляво на дясно, отгоре надолу, и не оставяш дупки.

Защо това е важно? Защото за всяко $n \in \mathbb{N}^+$ има **точно едно** попълнено двоично дърво с n върха. Това означава, че формата на дървото е напълно определена от броя върхове — много полезно свойство.

Теорема (височина): Височината на попълнено двоично дърво с n върха е $h = \lfloor \log_2 n \rfloor$.

Интуиция: Височината расте логаритмично, защото всяко ниво има двойно повече върхове от предишното. Това е ключово — всички операции по такова дърво ще ползват тази логаритмична височина.

1.2 Двоична пирамида (binary heap)

Определение 58. Двоична пирамида е попълнено двоично дърво, в което за всеки връх ключът му е **по-голям или равен** на ключовете на децата си (тогава е **максимална** пирамида), или е **по-малък или равен** (тогава е **минимална**).

В лекцията по подразбиране пирамида = max-heap. Обозначаваме това свойство „ключ на родителя $\geq$ ключ на детето“ с думата **пирамидално условие** (heap property).

Пример за пирамида:

```
    10
   /  \
  8    9
 /  \  /
3   2 1
```

Внимание! В пирамидата:

- максимумът е в корена;
- но **подредбата сред останалите не е сортирана** — пирамидата е много по-„хлабава“ структура от сортирания масив. По всеки път от корен към листо ключовете не растат, но между братя няма гарантиран ред.

Защо това е добро? Защото поддържането на сортиран масив струва $O(n)$ на вмъкване, докато поддържането на пирамида — само $O(\log n)$. Това е компромис между „пълнен ред“ (сортиран масив) и „пълнен хаос“ (несортиран масив). Пирамидата е средата — *достатъчно* подредба, за да си вадим максимума бързо, и *достатъчно* гъвкавост, за да правим вмъкване бързо.

1.3 Реализация чрез масив

Тук става интересно. **Не пазим пирамидата като дърво от обекти с указатели** — пазим я като обикновен масив $A[1 \dots n]$, в който $A[i]$ е ключът на върха с адрес i (адресите се присвояват от корена надолу, ляво на дясно). Свойствата:

Връх с индекс i	Свързан връх	Индекс
$A[i]$	Родител	$\lfloor i/2 \rfloor$
$A[i]$	Ляво дете	$2i$
$A[i]$	Дясно дете	$2i + 1$

Защо това работи? Защото попълненото дърво няма дупки — линеаризацията по нива (от ляво на дясно) ни дава „плътен“ масив без пропуски. Затова всеки преход родител↔дете е едно умножение или деление с 2.

Кои индекси отговарят на листа? Всеки $i > \lfloor n/2 \rfloor$ — половината от върховете са листа. Това ще се окаже изключително важно за анализа на сложността по-късно.

2. Наивно построяване на пирамида

Задачата: Даден е произволен масив $A[1 \dots n]$. Искаме да го размените така, че да стане пирамида.

2.1 Идеята

Най-естественият подход: пази инвариант, че подмасивът $A[1 \dots i - 1]$ е пирамида, и „прибавяй“ следващия елемент към края, поправяйки нещата нагоре.

Когато добавим $A[i]$, дървото $A[1 \dots i]$ може и да не е пирамида, защото новият връх може да е по-голям от родителя си. Но забележи едно много специално свойство: **нарушенията могат да са само между новия връх и негови предци по пътя нагоре** (това е „почти-пирамида“ в Определение 62 от лекцията).

2.2 Псевдокод

```
Алгоритъм 6: Naive Build Heap
Naive_Build_Heap(A[1..n])
1   for i ← 2 to n
2       k ← i
3       while A[Parent(k)] < A[k] do
4           swap(A[k], A[Parent(k)])
5           k ← Parent(k)
```

Стратегията се нарича „изплуване нагоре“ (**bubble-up / sift-up**): новият елемент „изплува“ към корена, докато не намери родител, който е по-голям или равен на него, или докато сам не стане корен.

2.3 Защо това е коректно (с интуиция)

Доказателството е с **инвариант на цикъла**:

Инвариант: Всеки път, когато изпълнението е на ред 1, $A[1 \dots i - 1]$ е пирамида.

База ($i = 2$): $A[1 \dots 1]$ е едноелементен масив — тривиално е пирамида.

Поддръжка: Допускаме, че $A[1 \dots i - 1]$ е пирамида. Добавянето на $A[i]$ ни дава „почти-пирамида“ — всички инверсии са по пътя от $A[i]$ до корена. While-цикълът систематично разменя всеки нов елемент с родителя му, докато не престане да има инверсии. Ключово наблюдение: **всяка такава размяна премахва една инверсия и не въвежда нова**, защото елементът, който отива нагоре, е по-голям от и двамата си бивши братя и сестри (бил е по-голям от родителя, който беше техен общ родител и беше $\geq$ от тях). Следователно while-цикълът терминира за най-много $\lfloor \log_2 i \rfloor$ стъпки (височината на дървото), и след излизането му $A[1 \dots i]$ е пирамида.

Терминация: При $i = n + 1$ инвариантът става „ $A[1 \dots n]$ е пирамида“.

2.4 Сложност по време

Външният цикъл прави $n - 1$ итерации. Във вътрешния цикъл, в най-лошия случай, всеки нов елемент изплува до корена — това са $\lfloor \log_2 i \rfloor$ размени за стъпка i . Тогава:

$$T(n) \in \Theta\left(\sum_{i=2}^n \log_2 i\right) = \Theta(\log_2 n!) = \Theta(n \log n)$$

(използваме формулата $\log n! \sim n \log n$).

Извод: Наивното построяване работи за $\Theta(n \log n)$. Това не е лошо, но ще видим, че **може и по-бързо** — линейно време!

💡 **Интуиция защо е $n \log n$:** Долният ред (листата) са най-многобройни — половината от всички върхове. И именно те трябва да изплуват в най-дълъг път (от листо до корен — височина $\log n$). Тоест $\frac{1}{2}$ от върховете правят $\log n$ работа. Това е $\Theta(n \log n)$. Това наблюдение ще е критично за разбирането защо **бързото** построяване работи обратното — там много върхове правят малко работа, защото листата изобщо не се пипат.

3. Бързо построяване на пирамида: Build Heap с Heapify

Идеята тук е **обратната посока**: вместо да добавяме от ляво на дясно (от корен надолу), ще обхождаме **отдясно наляво** (от листата към корена).

3.1 Помощната функция Heapify

Преди Build Heap имаме нужда от една по-обща операция. Представи си следната ситуация:

- Имаш индекс i в масив A .
- Лявото поддърво $A[\text{Left}(i)]$ и дясното поддърво $A[\text{Right}(i)]$ са вече пирамиди.
- Но $A[i]$ може да е по-малък от децата си — тоест $A[i]$ като цяло **може и да не е пирамида**.

Heapify(A, i) поправя именно тази ситуация: премества $A[i]$ надолу по дървото, докато не намери позиция, в която е $\geq$ от децата си, или докато не стане листо.

Стратегията „размени с по-голямото дете“

На всяка стъпка избираме **по-голямото от двете деца** на текущата позиция и го разменяме с текущия елемент, ако то е по-голямо от него.

Защо точно с по-голямото дете? Защото след размяната този, който отива нагоре, трябва да е $\geq$ от *двете* си нови деца. Ако разменим с по-малкото дете, то ще се окаже над по-голямото — нова инверсия!

Итеративен псевдокод

```
Алгоритъм 7: Iterative Heapify
Iterative_Heapify(A[1..n], i)
// Допускане: A[[Left(i)]] и A[[Right(i)]] (ако съществуват) са пирамиди.
1  j ← i
2  while j ≤ ⌊n/2⌋ do
3      left ← Left(j)
4      right ← Right(j)
5      if A[left] > A[j]
6          largest ← left
7      else
8          largest ← j
9      if right ≤ n and A[right] > A[largest]
10         largest ← right
11     if largest ≠ j
12         swap(A[j], A[largest])
13         j ← largest
14     else
15         break
```

Защо е коректно (интуиция)

Доказателството е с инвариант, но идеята е проста: на всяка стъпка имаме един „неправилно поставен“ елемент, който се движи надолу. Поддърветата от двете му страни вече са пирамиди (по предположение). При размяна с по-голямото от децата, елементът, който отива нагоре, става $\geq$ от двете си нови деца и затова пирамидалното условие в новата му позиция е изпълнено. Същият проблем — лошо поставен елемент — се „премества“ едно ниво по-надолу. Тъй като дървото е крайно, някога елементът става листо или намира своето място, и Heapify приключва.

Сложност по време на Heapify

Елементът може да слезе най-много с височината на поддървото — затова $\Theta(\log m)$, където m е броят елементи в поддървото $A[[i]]$. Това е *горната граница* — Heapify може и да приключи бързо, ако елементът „си е на мястото“ с малко стъпки.

3.2 Алгоритъмът `Build Heap`

```
Алгоритъм 8: Build Heap
Build_Heap(A[1..n])
1   for i ← [n/2] downto 1
2       Heapify(A, i)
```

Тривиално кратко! Но защо работи и защо започваме от $\lfloor n/2 \rfloor$?

Защо започваме от $\lfloor n/2 \rfloor$?

Защото всеки индекс $i > \lfloor n/2 \rfloor$ съответства на **листо**, а листата вече са тривиални пирамиди (всеки самостоятелен връх е пирамида). Няма какво да правим с тях — просто ги пропускаме.

Защо вървим отдясно наляво (downto)?

Защото `Heapify(A, i)` изисква **поддърветата на i вече да са пирамиди**. Ако вървим от листата нагоре, то когато стигнем индекс i , всички индекси с по-голяма стойност (включително $2i$ и $2i + 1$) вече сме обработили. Гарантирано е, че техните поддървета са пирамиди.

3.3 Доказателство за коректност

Теорема 67. За всеки вход A , `Build Heap` построява пирамида от него.

Инвариант 16: Всеки път, когато изпълнението е на ред 1, $A[\lfloor i + 1 \rfloor], A[\lfloor i + 2 \rfloor], \dots, A[n]$ са пирамиди.

База: $i = \lfloor n/2 \rfloor$. Всички индекси $> \lfloor n/2 \rfloor$ са листа, а всяко листо е тривиална пирамида. ✓

Поддръжка: Допускаме, че инвариантът държи за някое i . Според инварианта, $A[\text{Left}(i)]$ и $A[\text{Right}(i)]$ са пирамиди (защото $\text{Left}(i) > i$ и $\text{Right}(i) > i$). Затова можем коректно да приложим `Heapify(A, i)`, и след това $A[i]$ става пирамида. Сега имаме, че $A[i], A[\lfloor i + 1 \rfloor], \dots$ са пирамиди. След декрементиране на i , инвариантът отново държи. ✓

Терминация: При $i = 0$, имаме че $A[1], \dots, A[n]$ са пирамиди. Но $A[1] = A$. ✓

⚠ **Тънкост в инварианта.** Авторът специално подчертава, че инвариантът **не може** да се формулира само като „ $A[2i]$ и $A[2i + 1]$ са пирамиди“. Защо? Защото при декрементиране на i , новото $2i$ и новото $2i + 1$ са различни индекси от старите $2i$ и $2i + 1$ — и инвариантът няма да оцелее. Затова формулираме инварианта по-силно: **всички** $A[k]$ за $k > i$ са пирамиди.

3.4 Сложност по време: линейна!

Теорема 68. `Build Heap` работи за $\Theta(n)$.

Защо това е изненадващо? Викаме `Heapify` $\Theta(n)$ пъти, и `Heapify` работи за $O(\log n)$ — наивната оценка би била $O(n \log n)$. Но истината е по-фина.

Ключова идея: `Heapify` струва не $\log n$ за всеки връх, а толкова, колкото е височината на поддървото на този връх. И тук работят два факта:

1. **Половината от върховете са листа (височина 0)** — `Heapify` изобщо не се вика върху тях.
2. **Малко върхове са на голяма височина** — например само 1 връх е на височина $\log n$ (коренът), 2 върха са на височина $\log n - 1$, и т.н.

Точното разсъждение използва, че **броят на върховете с височина k в попълнено дърво е $\left\lceil \frac{n/2^k}{2} \right\rceil$** . Тогава:

$$T(n) \leq \sum_{k=1}^{\lfloor \log n \rfloor} \left\lceil \frac{n/2^k}{2} \right\rceil \cdot \Theta(k) = \Theta \left(\sum_{k=1}^{\lfloor \log n \rfloor} \frac{n}{2^k} k \right)$$

Сега ключът — редът $\sum_{k=1}^{\infty} \frac{k}{2^k}$ е **сходящ** (с критерия на д'Аламбер: $\lim \frac{a_{k+1}}{a_k} = \frac{1}{2} < 1$), и сборът му е константа (равна на 2). Затова:

$$T(n) \leq n \cdot \underbrace{\sum_{k=1}^{\infty} \frac{k}{2^k}}_{\text{константа}} = \Theta(n)$$

💡 **Интуиция в едно изречение:** Колкото по-високо е едно поддърво (тоест колкото по-скъпа е `Heapify` там), толкова **по-малко** върхове има на тази височина. Двете неща балансират и дават линейно общо време. Това е огромно подобрение пред $n \log n$ на наивното построяване.

Част II. Сортиращ алгоритъм HEAPSORT

4. Идеята на HEAPSORT

Имаме структура (пирамида), от която можем да извадим **максимума** за $O(1)$ (той винаги е в корена). Това веднага подсказва стратегия за сортиране:

1. Построй пирамида от масива.
2. **Извади** максимума, постави го **в края** на масива.
3. Размерът на пирамидата намалява с 1. Възстанови я с `Heapify`.
4. Повтори, докато пирамидата има само 1 елемент.

Накрая масивът ще е сортиран **възходящо**, защото слагаме максимумите от ляво на дясно, от края към началото.

Това е „in-place“ сортиране — не ни трябва допълнителна памет, само самият масив.

5. Псевдокод

```
Алгоритъм 9: Heapsort
Heapsort(A[1..n])
1  Build_Heap(A)
2  for i ← n downto 2
3      swap(A[1], A[i])
4      Heapify(A[1..i-1], 1)
```

Какво се случва на всяка итерация:

- $A[1]$ е максимумът — разменяме го с $A[i]$ и сега максимумът е на правилното си място, в края.
- Размерът на „активната“ пирамида намалява с 1: тя е $A[1 \dots i - 1]$.
- $A[1]$ обаче вече не е $\geq$ от децата си — извикваме `Heapify(A[1..i-1], 1)` за да поправим пирамидата. Останалите две деца $A[2]$ и $A[3]$ вече са корени на пирамиди (тяхното поддърво не е било пипано), затова `Heapify` е приложима.

6. Доказателство за коректност

Теорема 69. Heapsort е коректен сортиращ алгоритъм.

Инвариант 17: Всеки път, когато изпълнението е на ред 2:

1. Текущият $A[i + 1 \dots n]$ се състои от $n - i$ най-големите елементи на първоначалния масив, **в сортиран вид**.
2. Текущият $A[1 \dots i]$ е пирамида.

База ($i = n$): $A[n + 1 \dots n]$ е празен — вакуумно отговаря на „0 най-големи в сортиран вид“. $A[1 \dots n]$ е пирамида според Теорема 67. ✓

Поддръжка: Допускаме инварианта. От втората част, $A[1]$ е максимум на $A[1 \dots i]$.

След размяната на ред 3, $A[i \dots n]$ съдържа $n - i + 1$ най-големи елемента в сортиран вид. Това утвърждава първата част на инварианта спрямо новата стойност на i .

За втората част: след размяната, $A[1 \dots i]$ може да не е пирамида (нов елемент дойде в корена), **но поддърветата на корена $A[2]$ и $A[3]$ си остават пирамиди** (не са пипани!). Heapify на ред 4 възстановява пирамидалното условие. Спрямо новото i , инвариантът държи. ✓

Терминация ($i = 1$): $A[2 \dots n]$ съдържа $n - 1$ най-големи елемента в сортиран вид. Значи $A[1]$ е минимумът. Тогава целият $A[1 \dots n]$ е сортиран. ✓

 **Интуиция:** Във всяка итерация „вадим“ следващия по големина елемент и го слагаме в неговото окончателно място. Пирамидата изиграва ролята на „черна кутия“, от която можем да вадим максимума бързо.

7. Сложност по време

- **Build Heap** на ред 1: $\Theta(n)$.
- **For-цикълът** на редове 2–4: $n - 1$ итерации, всяка с по една размяна ($\Theta(1)$) и едно **Heapify** ($\Theta(\log i)$).

Тогава:

$$T(n) = \Theta(n) + \sum_{i=2}^n \Theta(\log i) = \Theta(n) + \Theta(n \log n) = \Theta(n \log n)$$

Сложност по памет: $\Theta(1)$ за итеративния вариант на Heapsort (няма допълнителни масиви), $\Theta(\log n)$ при рекурсивния вариант (заради стека на рекурсията).

8. Свойства на HEAPSORT (хубаво да се знаят)

- **In-place** — да.
 - **Стабилен** — **не!** Лесен контрапример: вход от еднакви елементи. `Build Heap` не променя нищо, но първата размяна `swap(A[1], A[n])` ще премести някой елемент от началото в края — нарушава се началната подредба на равни елементи.
 - **Най-добър случай:** пак $\Theta(n \log n)$ — за разлика от Insertion Sort, който при сортиран вход е $\Theta(n)$. Heapsort винаги си плаща пълната цена.
-

Част III. MERGESORT като пример за „Разделяй-и-Владей“

9. Какво е „Разделяй-и-Владей“?

Това е алгоритмична схема с три фази:

1. **Разделяй** (Divide): разбий задачата на по-малки подзадачи.
2. **Владей** (Conquer): реши подзадачите (обикновено рекурсивно).
3. **Комбинирай** (Combine): обедини решенията на подзадачите в решение на цялата задача.

В MERGESORT акцентът е върху **третата фаза** — там става истинската работа.

10. Идеята на MERGESORT

- **Разделяй:** Раздели масива $A[1..n]$ на две половини $A[1..n/2]$ и $A[n/2 + 1..n]$.
- **Владей:** Сортирай рекурсивно всяка половина.
- **Комбинирай:** **Слей** двата сортирани подмасива в един сортиран масив.

Спирачката на рекурсията: ако подмасивът има 1 елемент, той вече е сортиран.

 **Интуиция:** Половина от сортиран + половина от сортиран = цял несортиран? Не! **Сливането** е ключът. Ако имам две сортирани редици, мога да ги слея в линейно време, защото на всяка стъпка просто избирам по-малкия от двата „текущи“ елемента. Това превръща ефективно ред в по-голям ред.

11. Функцията `Merge` — сърцето на алгоритъма

```
Алгоритъм 12: Merge
Merge(A[1..n], l, mid, h)
// Допускане: A[l..mid] и A[mid+1..h] са сортирани.
1  n1 ← mid - l + 1
2  n2 ← h - mid
3  създай L[1..n1+1] и R[1..n2+1]
4  L[1..n1] ← A[l..mid]
5  R[1..n2] ← A[mid+1..h]
6  L[n1+1] ← ∞           // sentinel (пазач)
7  R[n2+1] ← ∞           // sentinel (пазач)
8  i ← 1
9  j ← 1
10 for k ← l to h
11     if L[i] ≤ R[j]
12         A[k] ← L[i]
13         i++
14     else
15         A[k] ← R[j]
16         j++
```

Идеята: Копираме двете половини в спомагателните масиви L и R . В края на всеки от тях слагаме „пазач“ ∞ — това е псевдо-елемент, който е по-голям от всичко истинско. Така можем безопасно да преглеждаме индексите без отделни проверки за „изчерпване“ на единия масив.

После просто сравняваме $L[i]$ и $R[j]$, и по-малкият отива на $A[k]$.

Защо при равенство ($L[i] \leq R[j]$) предпочитаме L ? Защото това прави Merge **стабилен** — елементите от лявата половина (които първоначално са били вляво в A) запазват своя относителен ред.

Коректност на Merge

Теорема 72. При допускане, че $A[l..mid]$ и $A[mid + 1..h]$ са сортирани, след работата на Merge, $A[l..h]$ съдържа точно техните елементи, в сортиран вид.

Доказателство с **инвариант на цикъла**:

Инвариант 19: Всеки път, когато изпълнението е на ред 10:

- $A[l..k - 1]$ съдържа $k - l$ най-малки елемента на L и R , в сортиран вид.
- $L[i]$ и $R[j]$ са най-малките елементи в L и R , които още не са копирани в A .

Лема 34 (важна!): На ред 11 не може едновременно $L[i]$ и $R[j]$ да са ∞ . Доказателство по противоречие: ако и двете са ∞ , значи всички истински елементи вече са копирани, но тогава броят на копираните е $n_1 + n_2 = h - l + 1 > h - l$, което противоречи на $k \leq h$.

База ($k = l$): $A[l..l - 1]$ е празен — вакуумно вярно. $L[1]$ и $R[1]$ са най-малките по конструкция. ✓

Поддръжка: Без ограничение на общността, $L[i] \leq R[j]$. Според инварианта, $L[i]$ е най-малкият некопиран елемент в обединението. И $L[i] \geq$ всеки елемент на $A[l..k - 1]$ (защото те са по-малки от $L[i]$ — изпратени са преди него). Затова след $A[k] \leftarrow L[i]$, $A[l..k]$ е сортиран. Спрямо новото k и новото i , инвариантът държи. ✓

Терминация ($k = h + 1$): $A[l..h]$ съдържа $h - l + 1$ най-малки елемента в сортиран вид. Понеже L и R заедно съдържат точно входните $h - l + 1$ елемента плюс два ∞ , всички $h - l + 1$ истински елемента са в $A[l..h]$. ✓

Сложност на Merge: $\Theta(h - l + 1)$ — линейно от размера на сливаните части. Външният цикъл прави $h - l + 1$ итерации, всяка с $\Theta(1)$ работа.

12. Самият MERGESORT — псевдокод

```
Алгоритъм 13: Mergesort
Mergesort(A[1..n], l, h)
1  if l < h
2      mid ← ⌊(l+h)/2⌋
3      Mergesort(A, l, mid)
4      Mergesort(A, mid+1, h)
5      Merge(A, l, mid, h)
```

Първоначалното извикване е `Mergesort(A, 1, n)`.

13. Доказателство за коректност на MERGESORT

Теорема 73. Mergesort е коректен сортиращ алгоритъм.

⚠ **Важно!** Това **не** е доказателство с инвариант на цикъла, защото Mergesort е рекурсивен, не итеративен алгоритъм. Затова правим **индукция по разликата** $h - l$ (която изразява размера на текущия подмасив минус 1).

База ($h - l = 0$): $A[l..h]$ е едноелементен. Едноелементният масив е тривиално сортиран, и Mergesort не прави нищо в този случай ($l < h$ е невярно). ✓

Индуктивна стъпка ($h - l > 0$): Допускаме, че за всички рекурсивни извиквания с по-малка разлика, Mergesort работи коректно. Тогава редове 3 и 4 коректно сортират $A[l..mid]$ и $A[mid + 1..h]$. По Теорема 72 (коректност на Merge), след ред 5 $A[l..h]$ е сортиран. ✓

Терминация: Когато началното извикване приключи работа, $A[1..n]$ е сортиран. ✓

💡 **Защо индукция, а не инвариант?** Защото рекурсията генерира **дърво от извиквания**, а не линейна верига. Индукцията по размера на подзадачата е естественият начин да докажем твърдение за всеки възел от това дърво.

14. Сложност по време на MERGESORT

Сложността на Mergesort удовлетворява **рекурентното уравнение**:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Защо?

- $2T(n/2)$ — двете рекурсивни извиквания върху половин масив всяко;
- n — линейната работа за `Merge` (тя обхожда всички n елемента).

Решаване чрез Мастър Теоремата

Сравняваме с шаблона $T(n) = aT(n/b) + f(n)$:

- $a = 2, b = 2, f(n) = n$.
- $n^{\log_b a} = n^{\log_2 2} = n^1 = n$.
- $f(n) = \Theta(n^{\log_b a}) = \Theta(n)$ — **втори случай** на Мастър теоремата.

Заклучение: $T(n) = \Theta(n^{\log_b a} \log n) = \boxed{\Theta(n \log n)}$.

Интуиция чрез рекурсивно дърво

Можем да си представим работата на Mergesort като дърво:

- **Корен:** задача с размер n (работата по слят на корен е n).
- **2 деца** със задачи с размер $n/2$ (общо работа на това ниво: $2 \cdot n/2 = n$).
- **4 внуци** със задачи с размер $n/4$ (общо работа: $4 \cdot n/4 = n$).
- ...
- **На всяко ниво общата работа е n .**
- Височината на дървото е $\log_2 n$.

Затова **общата работа е $n \cdot \log_2 n$.**

15. Сложност по памет

Сложност по памет: $\Theta(n)$ — за спомагателните масиви L и R в Merge. (Може да се аргументира и $\Theta(n \log n)$, ако не освобождаваме памет от всяко ниво поотделно, но при стандартна реализация е $\Theta(n)$.)

Това е **минусът** на Mergesort спрямо Heapsort — Mergesort не е in-place.

16. MERGESORT е стабилен!

Защото на ред 11 в Merge има **нестрого неравенство** $L[i] \leq R[j]$ — при равенство взимаме от L , тоест от левия (по-ранен) масив. Това запазва относителната подредба на равни елементи. Това е причина мнозинството реални системни сортирания (например `Arrays.sort` за обекти в Java) да са варианти на Mergesort.

Част IV. Приложение на MERGESORT — намиране на броя на инверсии

17. Какво е инверсия?

Определение 65. Нека $A[1..n]$ е масив от цели числа. **Инверсия** в A е двойка индекси $\langle i, j \rangle$, такива че $1 \leq i < j \leq n$ и $A[i] > A[j]$.

С думи: двойка индекси, на които елементите са „в погрешен ред“ — по-големият е по-вяво.

Примери:

- $A = [5, 1, 3, 2, 4]$ — инверсиите са $\langle 1, 2 \rangle, \langle 1, 3 \rangle, \langle 1, 4 \rangle, \langle 1, 5 \rangle, \langle 3, 4 \rangle$ — общо 5.
- Сортиран масив — **0 инверсии**.
- Обърнат масив с различни елементи — $\binom{n}{2} = \frac{n(n-1)}{2}$ инверсии (максимум).

Защо ни интересува? Броят на инверсиите мери „колко неподреден“ е масивът. В машинното обучение, статистиката (коефициент на Кендъл τ), в сравняване на класирания — навсякъде.

Наивният алгоритъм работи за $\Theta(n^2)$:

```
Count_Inversions_Naive(A[1..n])
1  c ← 0
2  for i ← 1 to n-1
3      for j ← i+1 to n
4          if A[i] > A[j]
5              c++
6  return c
```


Целта: **алгоритъм за $O(n \log n)$.**

18. Стратегия „Разделяй-и-Владей“

Ако разделим A на L (лява половина) и R (дясна половина), всяка инверсия $\langle i, j \rangle$ попада в **точно една** от три категории:

1. **И двата индекса са в L** — „лява инверсия“.
2. **И двата индекса са в R** — „дясна инверсия“.
3. **i е в L , j е в R** — „смесена инверсия“.

Тогава:

$$\text{общо инверсии} = (\text{инверсии в } L) + (\text{инверсии в } R) + (\text{смесени инверсии})$$

Първите две се изчисляват **рекурсивно**. Третата е тази, която трябва да изчислим ефективно в **сливането**.

19. Ключовото наблюдение

Ако L и R са вече сортирани, можем да преброим смесените инверсии **по време на самото сливане**, без допълнителна работа!

Защо? Защото в момента, в който в Merge сравним $L[i]$ с $R[j]$ и установим, че $L[i] > R[j]$, то означава, че **всички елементи от $L[i..n_1]$ са по-големи от $R[j]$** (защото L е сортиран!). Тоест за всяко $k \geq i$, двойката $(L[k], R[j])$ е инверсия. Това са $n_1 - i + 1$ инверсии наведнъж.

💡 **Главната идея:** Сортирането **унищожава** инверсиите. Като сме умни, в процеса на сортиране можем **да ги преброим** преди да ги унищожим.

20. Псевдокод

Модифицирана функция `Merge-modified`

```
Merge-modified(A[1..n], l, mid, h)
// Допускане: A[l..mid] и A[mid+1..h] са сортирани.
1  n1 ← mid - l + 1
2  n2 ← h - mid
3  създай L[1..n1+1] и R[1..n2+1]
4  L[1..n1] ← A[l..mid]
5  R[1..n2] ← A[mid+1..h]
6  L[n1+1] ← ∞
7  R[n2+1] ← ∞
8  i ← 1
9  j ← 1
10 c ← 0 // брояч на смесени инверсии
11 for k ← l to h
12     if L[i] ≤ R[j]
13         A[k] ← L[i]
14         i++
15     else
16         A[k] ← R[j]
17         c ← c + n1 - i + 1 // ← ключовият ред!
18         j++
19 return c
```

Главната функция `Inversioncount`

```
Inversioncount(A[1..n], l, h)
1  if l ≥ h
2      return 0
3  else
4      mid ← ⌊(l+h)/2⌋
5      x ← Inversioncount(A, l, mid)
6      y ← Inversioncount(A, mid+1, h)
7      z ← Merge-modified(A, l, mid, h)
8      return x + y + z
```

Извиква се с `Inversioncount(A, l, n)`.

21. Доказателство за коректност

Структурата на алгоритъма е същата като на Mergesort — рекурсивно разделяй и слей. **Връща се число**, а не подреден масив (макар че между другото масивът се сортира).

Доказателството е по индукция по $h - l$.

База ($h - l \leq 0$): Едноелементен или празен подмасив има 0 инверсии. ✓

Индуктивна стъпка: Допускаме, че рекурсивните извиквания на редове 5 и 6 връщат коректно броя инверсии съответно в L и R . Остава да докажем, че `Merge-modified` връща точно броя на смесените инверсии.

Защо ред 17 е коректен?

При проверка на ред 12, разглеждаме два случая:

Случай 1: $L[i] \leq R[j]$.

- $\langle L[i], R[j] \rangle$ не е инверсия.
- Понеже R е сортиран, $R[j] \leq R[j + 1] \leq \dots \leq R[n_2]$. Затова и $\langle L[i], R[j + 1] \rangle, \langle L[i], R[j + 2] \rangle, \dots, \langle L[i], R[n_2] \rangle$ **не са инверсии**.
- Тогава броят инверсии между $L[i..n_1]$ и $R[j..n_2]$ е същият като броя инверсии между $L[i + 1..n_1]$ и $R[j..n_2]$.
- Затова просто инкрементираме i , **без да променяме c** . ✓

Случай 2: $L[i] > R[j]$.

- $\langle L[i], R[j] \rangle$ **е** инверсия.
- Понеже L е сортиран, $L[i] \leq L[i + 1] \leq \dots \leq L[n_1]$. Затова и $\langle L[i + 1], R[j] \rangle, \langle L[i + 2], R[j] \rangle, \dots, \langle L[n_1], R[j] \rangle$ **също са инверсии**.
- Общо това са $n_1 - i + 1$ инверсии, които включват $R[j]$.
- Сега инкрементираме j — повече никога няма да гледаме $R[j]$, така че трябва **веднъж завинаги** да отчетем неговите инверсии, което прави ред 17: $c \leftarrow c + (n_1 - i + 1)$. ✓

В края на `Merge-modified`, c съдържа точно броя на смесените инверсии. Тогава ред 8 на `Inversioncount` връща точния общ брой. ✓

💡 **Главната идея още веднъж:** Когато тазивам елемент от R преди някои останали елементи от L , той „прескача“ всички тях — и това са точно инверсиите. Колко са те? $n_1 - i + 1$ — броят на елементите в L , които още не са копирани.

22. Сложност по време

Рекурентното уравнение е същото като при Mergesort:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

(Защото `Merge-modified` прави абсолютно същата работа като `Merge`, плюс константно повече работа на ред 17 — което не променя асимптотиката.)

Следователно $T(n) = \Theta(n \log n)$.

Сравнение: Наивният алгоритъм беше $\Theta(n^2)$. Нашият — $\Theta(n \log n)$. За $n = 10^6$ това е разликата между „довърши се за секунди“ и „довърши се за дни“.

Резюме: Когато какво да използваме?

Алгоритъм	Време	Памет	In-place	Стабилен	Бележки
Heapsort	$\Theta(n \log n)$	$\Theta(1)$	✓	✗	Винаги $\Theta(n \log n)$, без значение от входа
Mergesort	$\Theta(n \log n)$	$\Theta(n)$	✗	✓	Базата на много реални имплементации; работи добре с външна памет
Insertion Sort	$\Theta(n^2)$ най-лош, $\Theta(n)$ най-добър	$\Theta(1)$	✓	✓	Подходящ за малки масиви или почти сортирани
Naive Build Heap	$\Theta(n \log n)$	$\Theta(1)$	✓	n/a	По-бавен от Build Heap
Build Heap (Floyd)	$\Theta(n)$	$\Theta(1)$	✓	n/a	Линейно построяване на пирамида

Ключови принципи / морал

1. **Пирамидата = компромис.** Между „пълнен ред“ (сортиран масив) и „пълнен хаос“ (несортиран масив). Достатъчно ред да се вади max за $O(1)$, достатъчно гъвкавост вмъкване да е $O(\log n)$.
2. **Build Heap е линеен заради конкавността на разходите.** Малко елементи са с голяма височина — които струват скъпо; много елементи са листа — които не струват нищо. Сборът е $\Theta(n)$.
3. **Heapsort = повтаряй „извади максимум“.** Прост принцип, изграден върху мощна структура.
4. **Mergesort = Разделяй-и-Владей в най-чист вид.** Истинската работа е в сливането.
5. **Алгоритъм може да брои нещо „безплатно“ в процеса на работа** (като броят на инверсии в Mergesort). Сортирането унищожава инверсиите — а интелигентният брояч ги преброява преди да изчезнат.

6. Доказателствата на коректност:

- Итеративни алгоритми → **инвариант на цикъла** (база / поддръжка / терминация).
- Рекурсивни алгоритми → **индукция по размера на подзадачата**.